

SPM: SIMT programming on GPUs

Academic year 2025-2026

Prof. Massimo Torquati
massimo.torquati@unipi.it

Outline

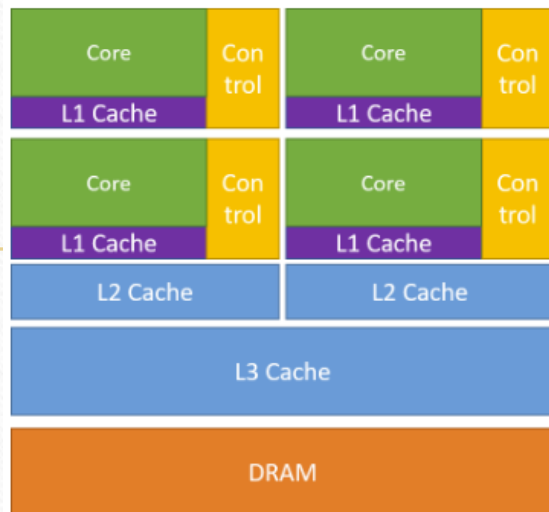
- GPU architecture and SIMT model
- Introduction to CUDA programming
- Simple examples



Introduction to GPUs

- **GPUs** (Graphical Processing Units) are designed for **massive parallelism** primarily targeting **data parallelism**
 - Today, the term GPU refers to **General Purpose GPU usage** (often called GPGPU), which are used beyond graphics purposes
- A GPU is a collection of a large number of GPU cores
 - The number of cores may range from hundreds to some thousands depending on the GPU model
 - Modern GPUs feature specialized cores designed for specific tasks (e.g., Ray Tracing Cores and Tensor Cores)
 - GPU cores are much simpler than CPU cores, targeting throughput instead of latency
- **SIMT (Single Instruction Multiple Threads)** execution model
 - Similar to SIMD at the hardware level
 - Many lightweight threads run the same kernel on different data
- **Programming model: Single Program Multiple Data (SPMD)**, many threads run the same kernel on different data

CPUs



- **CPUs:**

- a few powerful cores
- large caches
- high clock frequency
- sophisticated microarchitecture

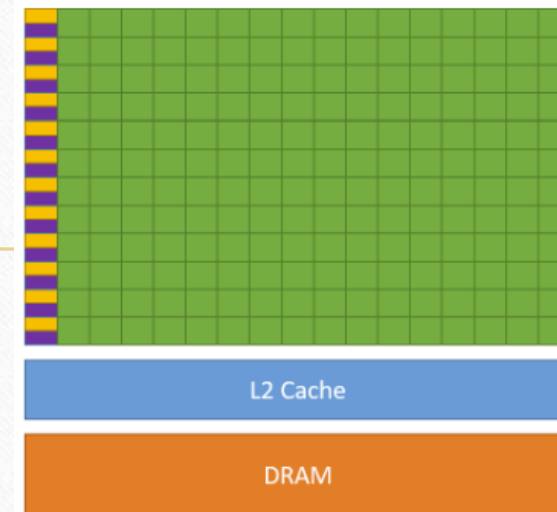
- **Optimized for low-latency tasks**

- Real-time decision making; Virtualized environments

- Intel Xeon Platinum 8571N (52 C)

- Rpeak **3.99 TFLOPs** (FP64), 307 GB/s, Base clock 2.4 GHz, TDP 300W

vs. GPUs



- **GPUs:**

- massive number of cores
- smaller caches per core (more emphasis on parallelism to hid memory latency)
- low clock frequency
- simpler microarchitecture (less emphasis on deep branch prediction and large caches)

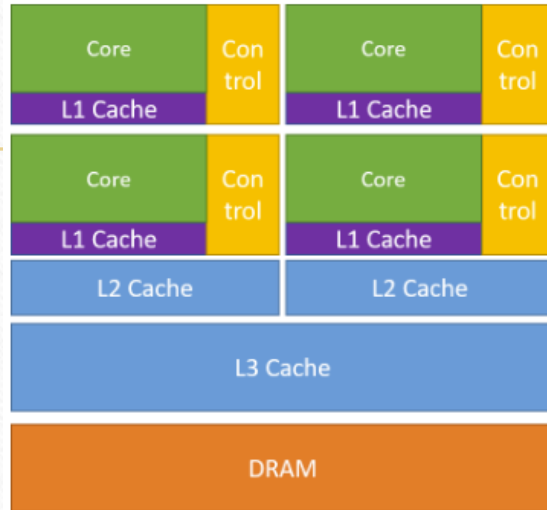
- **Optimized for high-throughput tasks**

- Computer Vision, AI training, Scientific computing

- NVIDIA H100 PCIe (16896 C + 528 TC)

- Rpeak **26 TFLOPs** (FP64), 2 TB/s, Base clock ~1 GHz, TDP 350W

CPUs



vs. GPUs



• SIMD

- SIMD executes a single instruction on multiple data elements in **lockstep** using vector units within a core
- Branching reduces SIMD efficiency

On GPUs, **branching is allowed**. The performance issue is **warp divergence**, when threads of the same warp take different branches

• SIMT

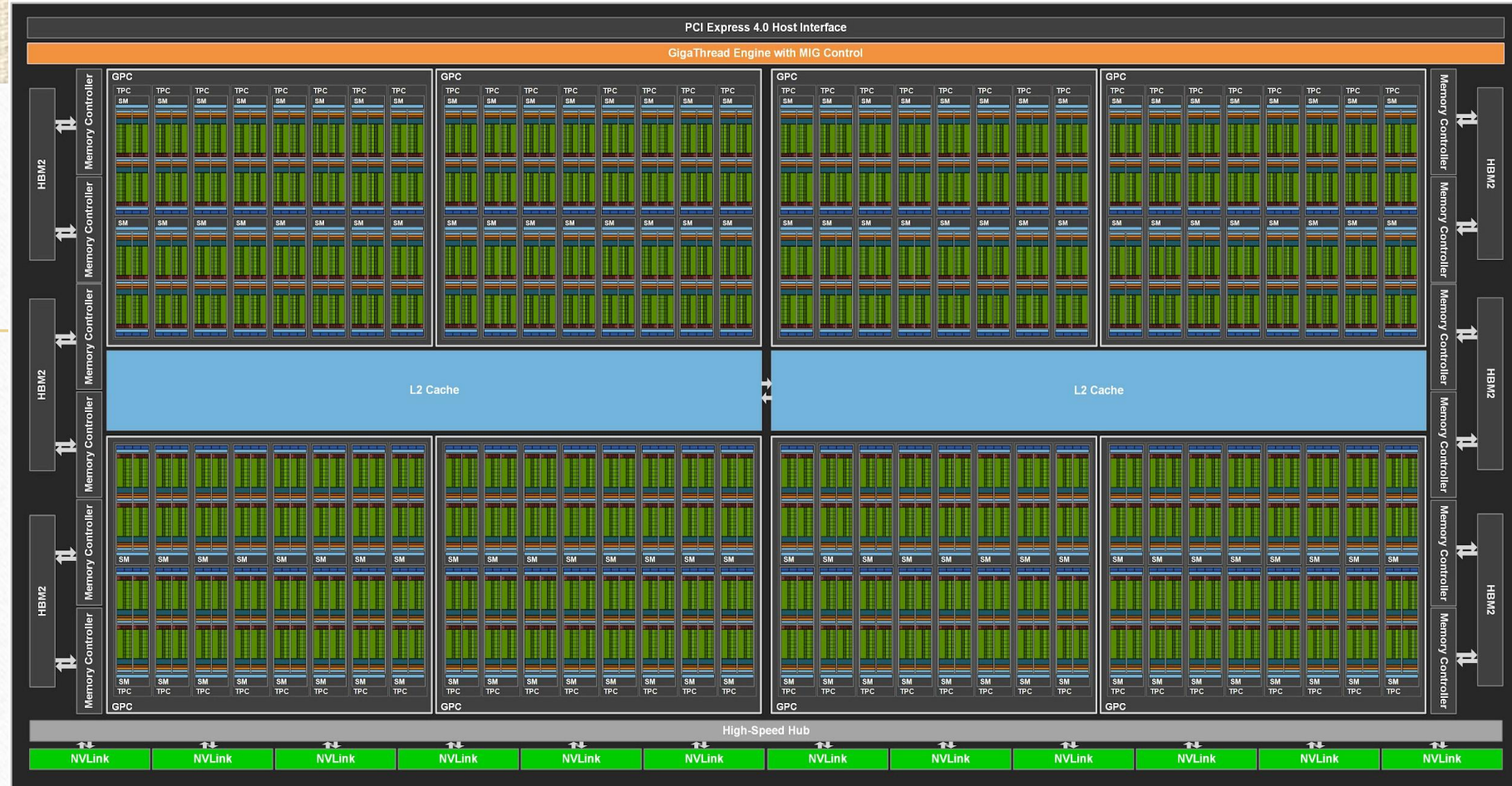
Within a **Streaming Multiprocessor (SM)**, threads are scheduled in groups called **warps**

- Warp execution mimics SIMD processing
- Threads in a warp execute the same instruction in **lockstep**
- At the programming model level, **individual threads in a warp are free to branch**, but divergence is serialized within the warp by masking inactive threads

SIMT execution model

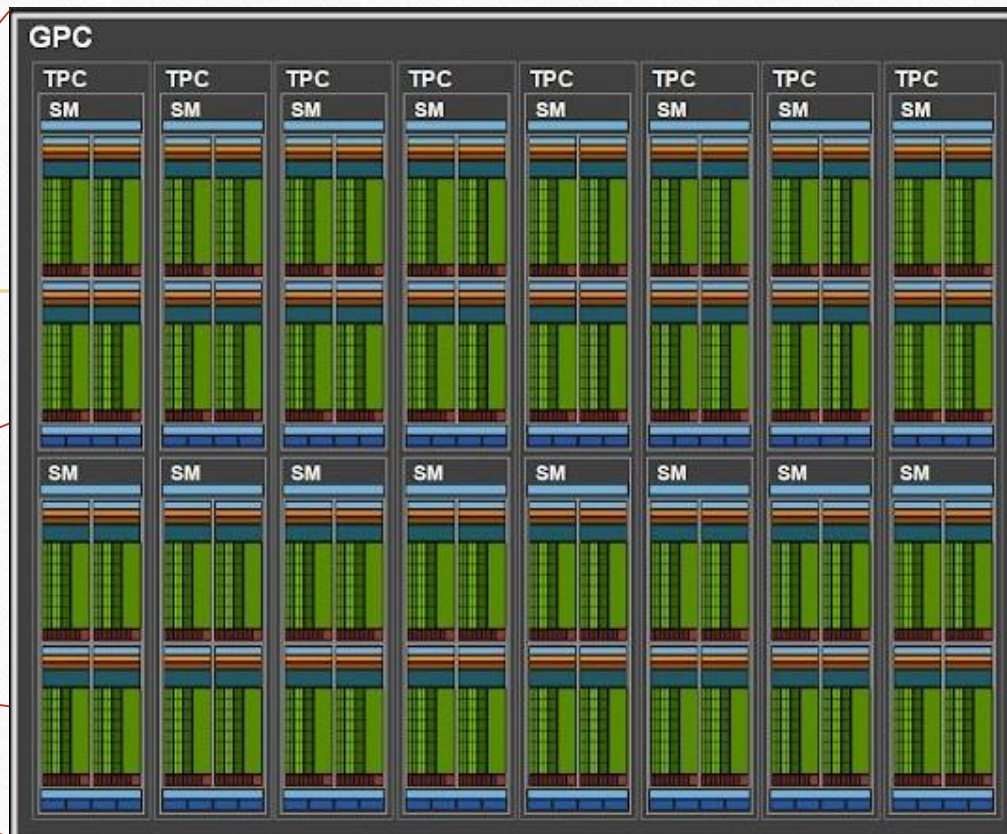
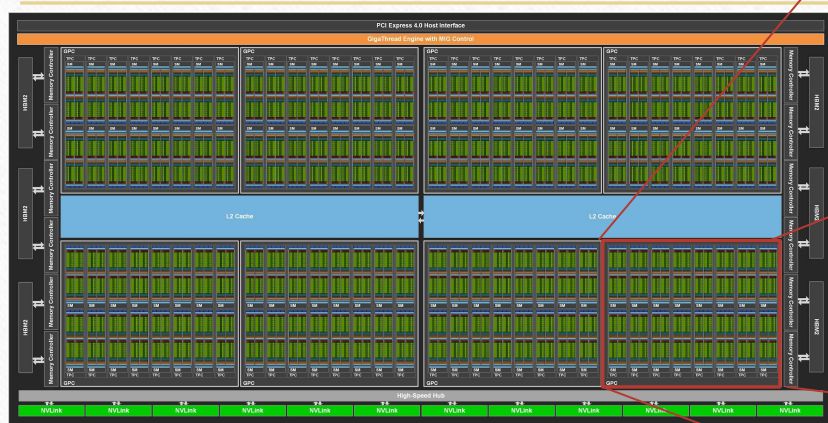
- Threads are organized in **warps** (32 threads). Warps are scheduled into a SM
 - Threads within a warp execute the same instruction in lockstep (when not diverged)
- When threads in a warp take different paths, **warp divergence** occurs, leading to inefficient execution, like branching issues in SIMD
 - The warp executes the branches one after the other (**masking inactive threads**), reducing efficiency
- Threads can also be grouped into larger units called **blocks**, which enable cooperation and data sharing among threads of the block
 - A block is partitioned into warps, and warps are scheduled as resources become available

NVIDIA GPUs



- A GPU comprises
 - some Graphics Processing Clusters (GPCs) each including
 - Several **Texture Processing Clusters (TPCs)** that provide core graphics functionalities
 - Several **Streaming Multiprocessors (SMs)**, the main compute units that execute warps
 - a complex memory hierarchy (constant memory, global memory, L2 cache, Shared Memory, L1 cache, and registers)
 - on-chip memory: registers, Shared Memory, L1 cache; off-chip memory: L2, global/DRAM

NVIDIA GPUs

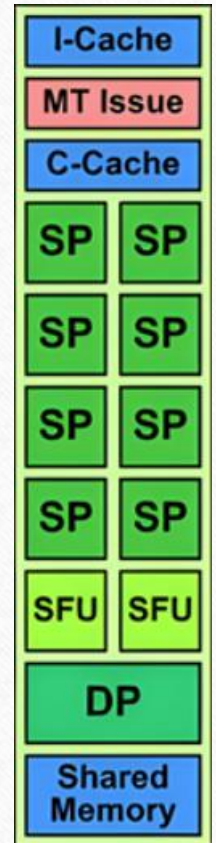


- Graphics Processing Cluster

- GPC is one of the top-level building block inside an NVIDIA GPU
- By grouping TPCs (each with multiple SMs), NVIDIA GPUs can scale to higher performance modularly
- Each GPC can work relatively independently to handle large parallel workloads

NVIDIA Streaming Multiprocessors (SMs)

- Each SM in a GPU integrates several components to execute a wide range of parallel processing tasks efficiently, among these:
 - **Streaming Processors** (SP cores): the basic execution unit in an SM. SPs execute warp instructions on different data elements
 - **Multi-Threaded Issue** (MT Issue): Handles the dispatch of instructions to various execution units within the SM. *It schedules multiple warps to hide latency and keep units busy*
 - **Double Precision units** (DP) and Special Function units (SFU)
 - **Shared Memory**: a fast, low-latency on-chip memory shared by threads within the same block (allocated on an SM)
- **Each SM runs independently and executes multiple warps concurrently to hide memory latency**
 - The GPU scheduler assigns distinct blocks of threads to different SMs



GPU Memory Types

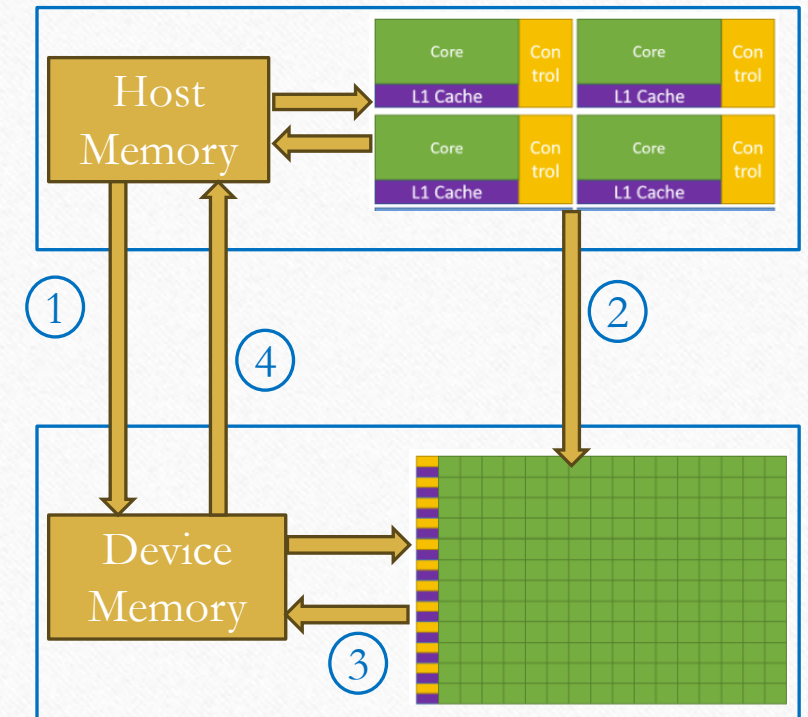
- **Registers** (large register file per SM): The fastest memory, **private to each thread**
 - If there are not enough registers, the compiler spills the extra values into **Local Memory**, which is physically allocated in **Global Memory**
 - **Warp scheduling, does not require CPU-style context switch.** Each active warp has its registers already allocated
- **Shared Memory** (48KB, 96KB,...): Low latency, shared between threads within a block
 - Often used as user-memory cache (i.e., scratchpad memory)
- **L2 cache** (1-8 MB): Cache of Global Memory, medium latency (slower than Shared Memory), GPU-wide access
- **Global Memory** (several GB): High latency (hundreds of clock cycles), accessible by all threads (GPU-wide)
 - **HBM** or **GDDR** technologies are used for Global Memory
- **Constant Memory** (64KB): Read-only GPU-wide, high latency (same base latency of Global Memory)
 - Its content is cached, thus making it very fast for uniform accesses (all threads reading the same location)
- **Texture Memory** (several GB): specialized for image processing
 - It is not extra memory. It is a different path to the Global Memory, with specialized caching

CUDA: Compute Unified Device Architecture

- **CUDA** is NVIDIA's parallel computing **platform and programming model** that allows developers to use GPUs for **general-purpose computing**
 - **OpenCL** is an open standard targeting multiple vendors, while CUDA is tightly integrated and optimized for NVIDIA GPUs
 - OpenCL is being adopted slowly mainly because of its verbosity and less efficiency on NVIDIA GPUs
- CUDA provides a software layer that **abstracts GPU hardware features**
- (Relatively) Low-level abstraction that matches the capabilities and performance characteristics of modern NVIDIA GPUs
 - Several tools build upon CUDA to simplify GPU programming (e.g., cuBLAS, Thrust, PyCUDA, ...)
- CUDA is one of the core technology used in many HPC applications
- It contributed to increase focus on energy efficiency (FLOPs/Watt)

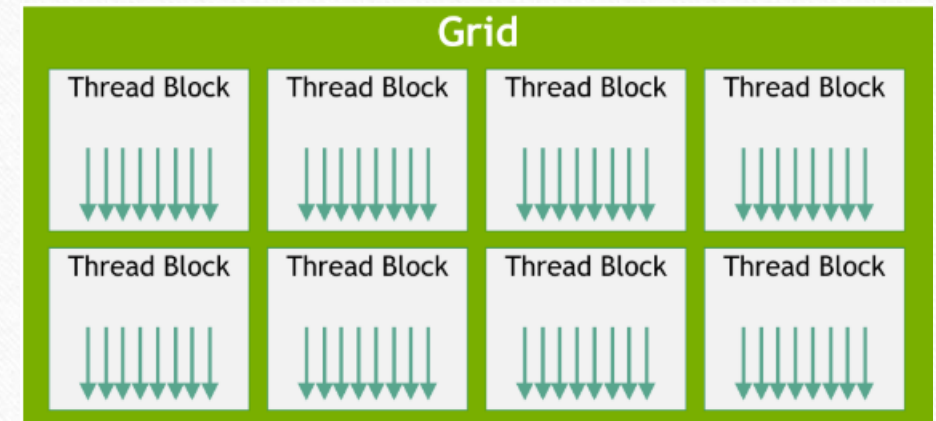
Basic concepts and terminology

- **Host:** The CPU and its memory hierarchy
- **Device:** The GPU and its memory hierarchy
- CUDA distinguishes between the **host program** (the one executed on the CPUs' cores) and **device program** (the one executed on the GPU)
- The device program comprises one or more functions called **kernels**
- The typical workflow between host and device programs is as follows:
 1. Host copies input data from CPU memory to GPU global memory
 2. Host launches device kernels, which may use internal GPU memory hierarchy
 3. Kernel(s) execute in parallel on the GPU; the host may wait for kernel(s) termination
 4. Host copies the result from GPU global memory to CPU memory or repeats from step 2



CUDA Execution Model

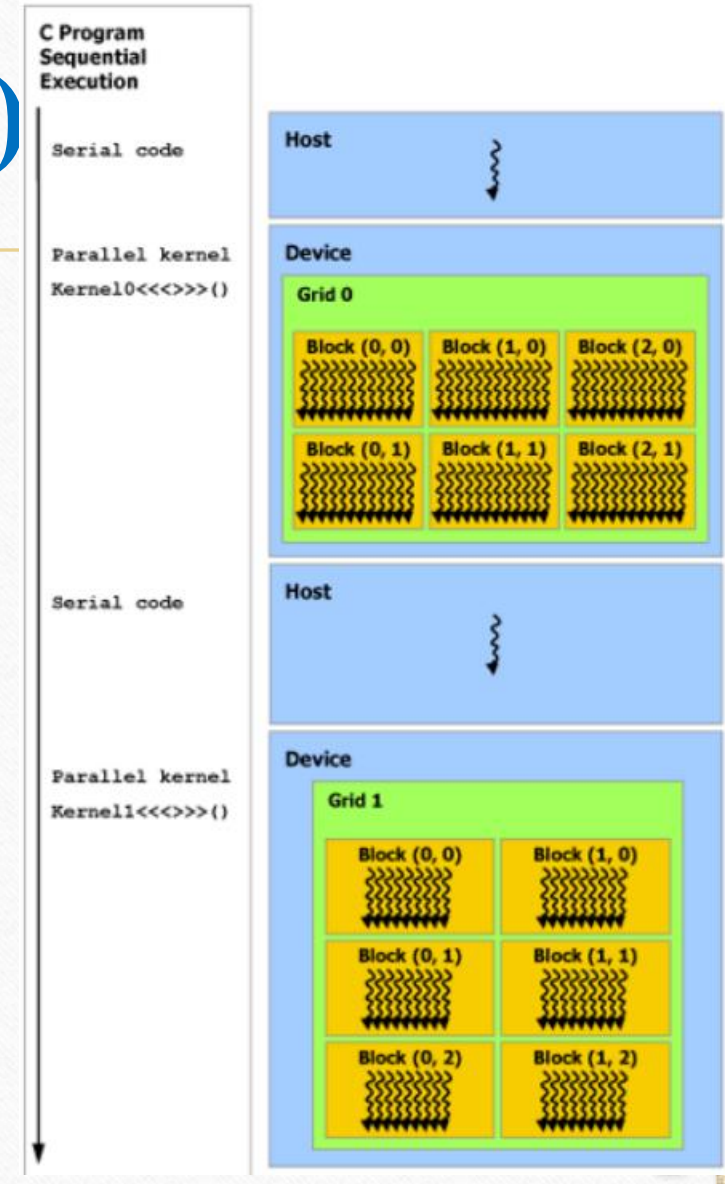
- CUDA extends C/C++ with language constructs and APIs to write GPU **kernels**
 - A GPU kernel is a function executed by many GPU threads in parallel
- CUDA uses a **hierarchical execution model**, where computations are split into **grids**, **blocks** and **threads**
 - A **thread** is the minimal unit of work
 - A group of threads forms a **thread block** (1D, 2D and 3D)
 - Thread blocks are organized into a **grid** (1D, 2D and 3D)
 - Each thread is identified by 1D, 2D, or 3D indices
 - Thread blocks are scheduled dynamically to SMs



Thread Block Cluster is a level of hierarchy introduced in NVIDIA CC 9 that groups thread blocks in clusters.

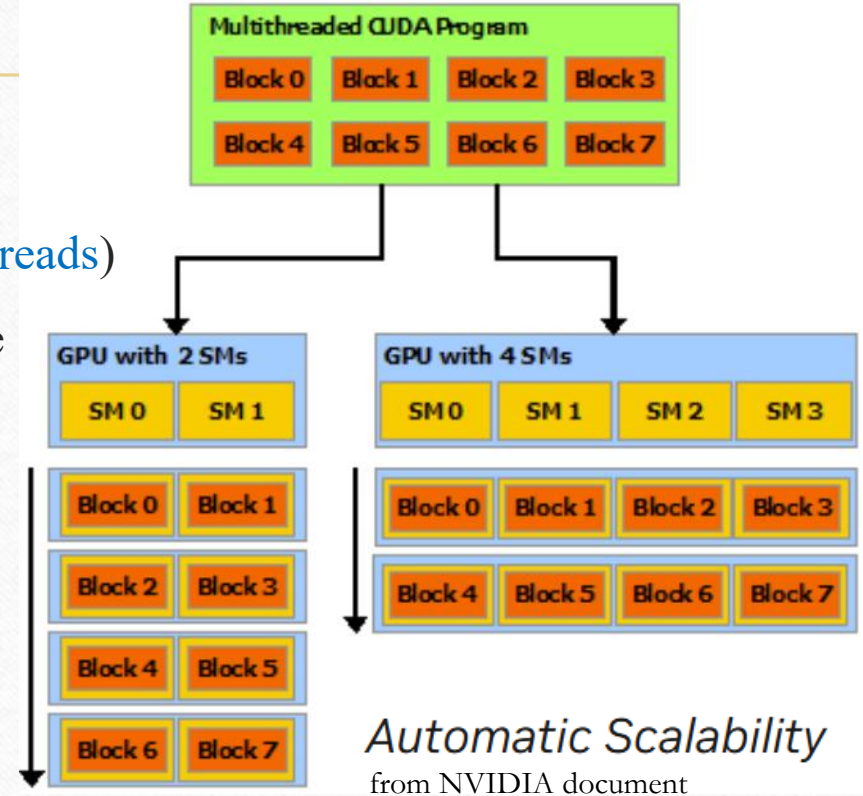
CUDA Execution Model (cont.)

- Kernels execute **asynchronously** with respect to the host program
 - Control returns to the CPU immediately after kernel launch
- The **kernels execute** according to the **SPMD model** (Single-Program Multiple-Data)
 - The host program loads the device program and starts its execution on the device
- **Distinct host and device address spaces**
 - Memory transfers (**Host-to-Device** and **Device-to-Host**) are explicitly managed by the host program
 - Kernel input/output are typically copied between host memory and device Global Memory
 - **CUDA Unified Memory** simplifies memory management with some performance cost
- The NVIDIA **nvcc** compiler compiles device code and invokes the host compiler for CPU code



CUDA Programming Model

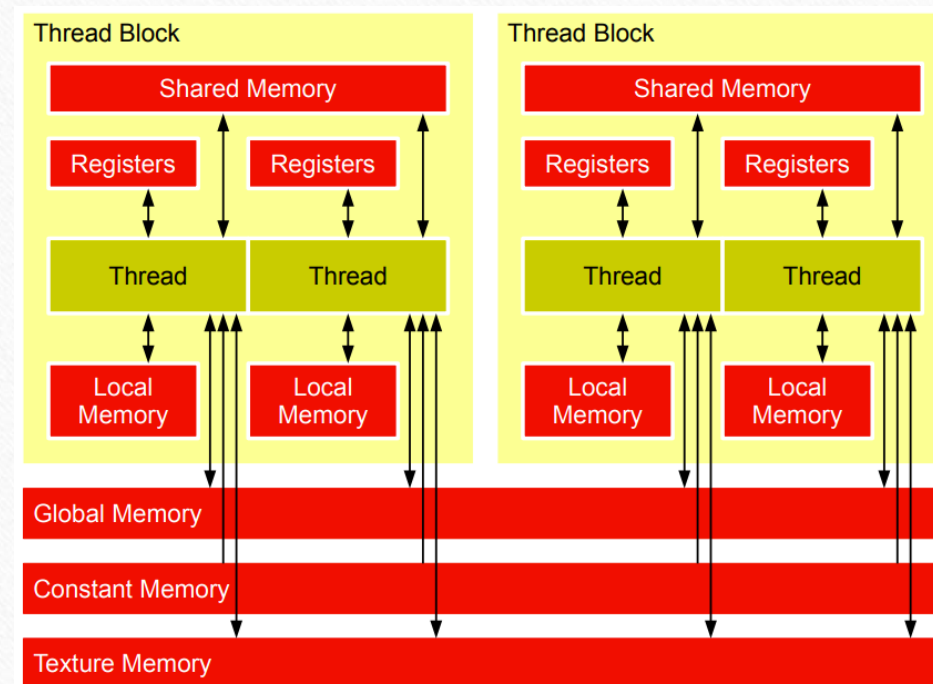
- CUDA provides
 - A **hierarchical** way to structure parallelism (**Grid** → **Blocks** → **Threads**)
 - **Shared-Memory abstraction**: threads within the same block can use shared-memory to cooperate on sub-problems
 - **Synchronization**: threads within a block can synchronize using barrier (**no implicit synchronization across blocks**)
 - **Automatic scalability**: blocks of threads are scheduled onto the available SMs and can be executed on GPUs with a different number of SMs without code changes



A GPU is built around an array of Streaming Multiprocessors (SMs) (see [Hardware Implementation](#) for more details). A multithreaded program is partitioned into blocks of threads that execute independently from each other, so that a GPU with more multiprocessors will automatically execute the program in less time than a GPU with fewer multiprocessors.

Kernels' Memory Abstractions

- Main address spaces visible to kernels
 - **Per-thread private memory** (**registers** / **local memory**)
 - Local memory is in Global Memory and used when registers are not enough or for large per-thread arrays
 - **Per-block Shared Memory**: threads within the same thread block can synchronize execution, and share access to shared memory
 - Requires explicit synchronization to avoid race conditions
 - **Device global memory**: each thread of all blocks can read and write **global memory**
 - Can also be accessed from the host through explicit copies or through CUDA **Unified Memory**
 - To coordinate across blocks, you typically synchronize via separate kernel launches, or cooperative groups on supported GPUs



Host-Device Synchronization

- The host program must wait for completion **before reading the results**
 - Remember: kernel launches are typically asynchronous with respect to the host
- **cudaDeviceSynchronize()** / **cudaStreamSynchronize()**
 - **cudaDeviceSynchronize** blocks until all previously issued work in the current context completes.
 - **cudaStreamSynchronize** blocks until work in that stream completes
- **CUDA Streams** provide flexibility in many aspects, and specific streams can be synchronized
 - A stream is essentially a queue of operations (kernels, memory copies, etc.) that execute in order on the device
 - By default, operations are issued to the “default CUDA stream”. Creating multiple streams enables different operations to run concurrently
 - Multiple streams can enable concurrency (subject to hardware/resources and dependencies)
 - You can **overlap** kernels with transfers using `cudaMemcpyAsync` and pinned host memory (when supported)
 - Selective synchronization with **cudaStreamSynchronize(stream)**
 - CUDA events for finer control are available (not covered here)

Thread synchronization

1/2

- All threads in a warp execute the same instruction in lockstep (SIMD-like execution)
- Warps on the same SM may belong to different blocks and, if concurrent kernels are supported, to different kernels
 - warp0 might belong to one kernel, while warp1 belongs to another kernel
- Different blocks can run concurrently on the same SM (if there are enough resources, e.g., registers)
- Within a block, threads can synchronize with **barriers**. **Atomic operations** can be used to coordinate accesses to Shared or Global Memory
- **CUDA-specific atomic operations**
 - Fine-grained access (**read-modify-write**) to a shared variable (e.g., atomicAdd, atomicCAS)
 - Useful to avoid race conditions when multiple threads concurrently update Shared or Global Memory
 - We will study atomic operations in C++; atomic operations in CUDA follow similar principles to C++, with GPU-specific scopes and constraints

Thread synchronization

2/2

- **Barriers**

- A barrier is a synchronization mechanism that forces a group of threads to wait until all threads in that group have reached the same point in the code
- `__syncthreads()`: a barrier for all threads within the same thread block
- `__syncwarp()`: a barrier for threads within the same warp (32 threads)
 - `syncwarp` accepts an optional `mask` argument to synchronize a subset of warp lanes
- Kernel completion is an ordering point in a stream: later operations in the same stream start only after the kernel finishes
- Inter-block synchronization is not available inside a normal kernel. Use separate kernel launches for global barriers, or cooperative groups (grid sync) on supported GPUs

Moving data to/from Device

- **cudaMemcpy()**: synchronous memory copy from host-to-device (H2D), device-to-host (D2H), or device-to-device (D2D)
 - The call blocks the host until the copy completes
- **cudaMemcpyAsync()**: asynchronous CUDA copy; returns control to the host immediately
 - The copy runs in a specified CUDA stream and can overlap with kernel execution if properly managed. Overlap requires pinned host memory (cudaHostAlloc/cudaMallocHost) and appropriate stream usage
 - **cudaStreamSynchronize()** or use CUDA events (query/timing)
- **Unified Memory** with **cudaMallocManaged()**, data can be accessed by both host and device without explicit copies (it uses on-demand page migration)
 - However, memory migration occurs automatically behind the scenes, which can introduce implicit synchronization points

Blocks and Threads

- Thread **blocks** can be scheduled in any order by the GPU scheduler
 - multiple blocks may run concurrently on one SM if there are resources (registers, shared memory)
 - blocks are assumed independent within a kernel (no implicit inter-block synchronization)
 - **if there are dependencies, use separate kernel launches** (or cooperative groups on supported GPUs)
- CUDA threads in a block execute the same kernel code (**SPMD**) and can cooperate via Shared Memory
 - The GPU schedules threads in **warps** of 32 threads
- **All warps in a thread block are scheduled onto the same SM**
- When all threads in a block complete, block resources become available for another block
 - **registers and Shared Memory are released, the next waiting block can then allocate resources on the SM to run the warps**

Grid and Blocks

- Grid: the collection of all thread blocks launched by a kernel call
- `dim3` is a CUDA struct type used to specify up to 3D values (x, y, z); used to set up grid and block dimensions
- Multi-dimensional thread and block indices are convenient for 2D, 3D problems

```
const int Nx = 12;  
const int Ny = 6;  
dim3 blockDim(4, 3);  
dim3 numBlocks( (Nx + blockDim.x - 1) / blockDim.x,  
                (Ny + blockDim.y - 1) / blockDim.y);
```

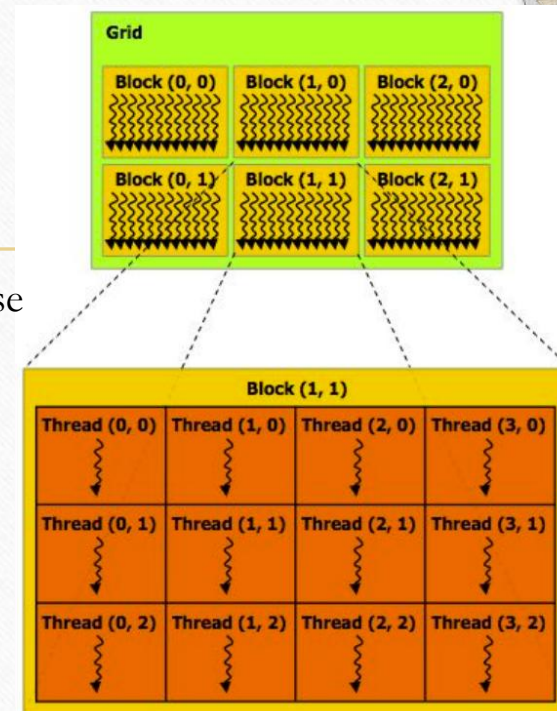
- **Index computation:** Inside the kernel, we can calculate the coordinates (x, y) using built-in variables:

```
int x = blockIdx.x * blockDim.x + threadIdx.x;  
int y = blockIdx.y * blockDim.y + threadIdx.y;
```

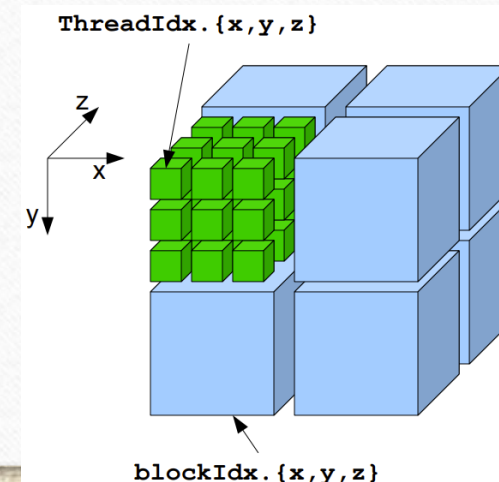
- Check boundaries when the grid may cover extra threads

```
if (x < Nx && y < Ny) <do-something>
```

2D case



3D case



CUDA kernel launch <<<X,Y>>>

- Threads are organized in a **grid** of **blocks**.
 - A grid is a collection of blocks.
 - A block is a group of threads with 1D, 2D, or 3D indices). Maximum 1024 threads per block

kernel function call

```
mykernel<<<X, Y>>>( kernel parameter list);
```

- The triple angle bracket defines a CUDA kernel call from host code
- It defines the grid configuration. In the 1D case:
 - X sets the number of blocks in the grid (gridDim.x);
 - Y sets the number of threads per block (blockDim.x)

NVIDIA CUDA GPU and CUDA version

- GPU Architecture:
 - NVIDIA Ampere (A30)
- PCIe 4.0 (x16) interface
- Peak throughput (theoretical)
 - FP32 10.3 TFLOPS
 - FP64 5.2 TFLOPS
- 24 GB of HBM2 (peak bandwidth 993 GB/s)
- TDP ~165W (930-1440 MHz clock frequency)
- 56 SM, 3584 CUDA cores
 - Each SM has 64 FP32 and 32 FP64 CUDA cores

CUDA toolkit 12.3, CUDA capability 8.0

```
torquati@node09:~$ nvcc --version
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2023 NVIDIA Corporation
Built on Wed_Nov_22_10:17:15_PST_2023
Cuda compilation tools, release 12.3, V12.3.107
Build cuda_12.3.r12.3/compiler.33567101_0
torquati@node09:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:   Ubuntu 22.04.5 LTS
Release:      22.04
Codename:     jammy
```

Host server: 2 AMD EPYC 7551 (64 cores 2-way SMT) at 2GHz

Compiling CUDA code

- nvcc orchestrates compilation for both the host and device program
- It produces a “**fat binary**”, which may contain multiple SASS images (sm_80, sm_90, ...), i.e., different GPU compute capability targets, and optionally **PTX** code as a fallback
 - PTX is a virtual GPU ISA that the NVIDIA driver JIT-compiles into native SASS for the installed GPU at runtime
- As usual -O2 -O3 control optimization levels
- --ptxas-options=-v : displays register usage, spill counts, and resource usage for debugging performance
- -arch=native generates SASS for the visible GPU (no PTX fallback). Use -gencode to compile for multiple targets
- Compiling with g++

```
nvcc myprog.cu -o myprog
```

```
g++ -c -O3 main.cpp -o main.o
nvcc -c -O3 kernels.cu -o kernels.o
nvcc main.o kernels.o -o myprog
```

```
nvcc -O3 myprog.cu -o myprog \
-gencode arch=compute_80,code=sm_80 \
-gencode arch=compute_90,code=sm_90 \
-gencode arch=compute_90,code=compute_90 // PTX
```


Vector addition in CUDA

Device code

```
// CUDA kernel for vector addition z= x + y
__global__ void add(int *x, int *y, int *z, int N) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx < N) // bound check
        z[idx] = x[idx] + y[idx];
}
```

build-in variables

`__global__` is a CUDA C++ qualifier defining a kernel function callable from the host

Host code

```
int main() {
    int N=1024*1024;           // Problem dimension
    int threadPerBlock = 128; // often set to a multiple of 32 (warp size)
    int numBlocks = (N + threadPerBlock-1)/threadPerBlock;
    ... // Allocate and initialize vectors on host and device
    ... // copy vector from host to device
    // Launch the kernel with total threads =threadPerBlock * numBlocks (≥N)
    add<<<numBlocks, threadPerBlock>>>(d_x, d_y, d_z, N);
    cudaDeviceSynchronize(); // ensure completion before using results
    ... // copy results back to host
}
```

Example: cuda/vsum.cpp

Declarations in CUDA

Function declaration	Execute on	Callable from
<code>__global__ float kernelFunction(...)</code>	Device	Host
<code>__device__ float deviceFunction(...)</code>	Device	Device
<code>__host__ float hostFunction(...)</code>	Host	Host

- The `__host__` qualifier is optional, functions without qualifiers are host by default
- The function with `__device__` qualifier can be called from device code only, typically inlined.
- It is possible to use both `__host__` and `__device__` qualifiers for the same function, the CUDA compiler produces two versions of the function, one for the device and one for the host
- `__device__ float dbuf[BLOCK_DIM];` declares a device global variable stored in global memory
 - `cudaMemcpyToSymbol()` and `cudaMemcpyFromSymbol()` should be used to copy data to/from `__device__` variables

CUDA SAXPY with Unified Memory 1/2

```
// SAXPY CPU kernel
void naive_SAXPY(const float *x, float *y,
                 const float a, int N) {
    for(int i=0; i<N; ++i)
        y[i] = a * x[i] + y[i];
}
```

- $A \times X + Y$: $i \in [0, N)$: $y[i] = a \cdot x[i] + y[i]$
- Parallelization with CUDA using one thread per vector element

```
// CUDA kernel for SAXPY y= a*x + y
__global__ void SAXPY(const float *x, float *y,
                     const float a, int N) {
    int i=blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) // bound check
        y[i] = a * x[i] + y[i];
}
```

- Threads access x and y with the same index, which gives **coalesced** global memory accesses
- **Coalescing**: Global memory is efficient when a threads in a warp access contiguous (and aligned) addresses

Examples:

cuda/saxpy.cpp

cuda/saxpy-noUM.cpp

Disclaimer: this is a naive SAXPY implementation. The code is not optimized! Optimized math kernels are offered by cuBLAS.

CUDA SAXPY with Unified Memory 2/2

- **Unified Memory (UM):** Memory accessible both from the host and the device in read/write mode
 - Data transfers to/from the device are handled automatically **using on-demand paging**
 - When a CPU or GPU thread accesses a page of memory that is not currently resident in the local memory, the UM driver generates a page fault and migrates that page to the physical memory of the requesting processor
- **Performance considerations:** While UM simplifies coding, frequent page faults can degrade performance due to the overhead of fault handling and page migration
 - Applications can use explicit prefetching (`cudaMemPrefetchAsync`) or memory advice hints (`cudaMemAdvise`) to optimize data placement

Example: cuda/saxpy-UM.cpp

```
#include<cuda_runtime.h>
...
int main() {
    int N= 1<<20; // Problem dimension
    float *x, *y;
    const float a = 2.5f;

    // Allocate Unified Memory for simplicity
    cudaMallocManaged(&x, N*sizeof(float));
    cudaMallocManaged(&y, N*sizeof(float));

    init(x, y); // initialize data
    int blockSize = 128; // should be a multiple of 32
    int numBlocks = (N + blockSize-1)/blockSize;

    // kernel executed with blockSize * numBlocks threads
    SAXPY<<<numBlocks, blockSize>>>(x, y, a, N);
    cudaDeviceSynchronize(); // Wait for kernel to finish

    cudaFree(x); cudaFree(y);
}
```

TO BE COMPLETED

Suggested Readings

- Chapter 7 of the Parallel Programming Concept and Practice course book
- «CUDA C++ Best Practices Guide» by NVIDIA Corporation, Jan 08, 2026
https://docs.nvidia.com/cuda/pdf/CUDA_C_Best_Practices_Guide.pdf
- Programming on Parallel Machines by Norm Matloff (Chapter 5)
https://heather.cs.ucdavis.edu/matloff/public_html/158/PLN/ParProcBook.pdf
- Additional resources:
 - «Programming Massively Parallel Processors: a hands-on approach», by W.W. Hwu, D.B. Kirk, I. El Hajj, Morgan Kaufmann, 4th ed.
 - «CUDA for Engineers, an introduction to high-performance parallel computing» by D. Sorti, M. Yurtoglu, Addison-Wesley